

Django

עבודה על פרוייקט מסכם

TomerBu

פרוייקט לסיום:

מאק:

```
django-admin startproject finalproject
```

```
cd finalproject
```

```
python3 -m venv env
```

```
source env/bin/activate
```

חלונות:

```
django-admin startproject finalproject
```

```
cd finalproject
```

```
python -m venv env
```

```
.\env\Scripts\activate
```

עבודה עם drf

api/views.py

```
from django.shortcuts import render
from rest_framework.response import Response
from rest_framework.request import Request

from rest_framework.decorators import api_view
# Create your views here.

@api_view(['GET'])
def say_hello(request: Request) -> Response:
    return Response({"message": "Hello, world!"})
```

api/urls.py

```
from api.views import say_hello
from django.urls import path

urlpatterns = [
    path('hello/', say_hello),
]
```

עבודה עם drf

<https://pypi.org/project/djangorestframework/>

1

```
pip install djangorestframework
```

2

```
python manage.py startapp api
```

finalproject/finalproject/settings.py

3

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'rest_framework',  
    'api',  
]
```

עבודה עם drf

finalproject/api/views.py/urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', include('api.urls')),
]
```

כדי שהכל יעבוד כשורה:
נריץ מיגרציות

```
python manage.py migrate
```

<http://localhost:8000/api/hello/>

בהורדה של הפרוייקט מגיט:

```
python -m venv env
```

```
.\env\Scripts\activate
```

```
pip install djangoRESTframework
```

```
python manage.py runserver
```

מודלים:

User

מייל
סיסמא

מה הכתובת למשלוח?

מחלקה להוספת פרטים

UserProfile

מחלקה קיימת בDjango

Please complete this online form to submit your request.

Title:	Dr	
First Name:		*
Last Name:		*
Company:		*
Address 1:		
Address 2:		
City:		*
County:		
Post Code:		
Telephone:		*
E-Mail:		*
Fax:		
What are you?	Customer	
Department to contact:	Sales	
Contact method:	Email	
Your request:		

מודלים:

```
from django.db import models
from django.contrib.auth.models import User

class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    bio = models.TextField(blank=True)
    birth_date = models.DateField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f'Profile of {self.user.username}'

    def __repr__(self):
        return f'UserProfile(user={self.user.username}, bio={self.bio}, birth_date={self.birth_date})'

class Tag(models.Model):
    name = models.CharField(unique=True, max_length=32)

    def __str__(self):
        return f'{self.name}'
```

python manage.py makemigrations api

python manage.py migrate

:Serializers

api/serializers.py

```
from api.models import UserProfile, Tag
from rest_framework.serializers import ModelSerializer

# json + create/update + validations
class UserProfileSerializer(ModelSerializer):
    class Meta:
        model = UserProfile
        fields = '__all__'
        # fields = ['bio', 'id']

class TagSerializer(ModelSerializer):
    class Meta:
        model = Tag
        fields = '__all__'
```

אחרי ההפסקה - נוסף את יתר המודלים
נוסף את יתר הסרייליזרים

ונכיר ModelViewSet

מודלים:

אפיון פרויקט בלוג עם משתמשים, כתבות ותגובות

User מחלקה מובנית בDRF

מבנה הנתונים (Models)

UserProfile מאפשר להוסיף פרטים על המשתמש

משתמשים (User)

- לכל משתמש תהיה הרשאה רגילה או הרשאת מנהל
- משתמשים רשומים יכולים לכתוב תגובות מנהלים יכולים ליצור, לערוך ולמחוק כתבות.

Post

Tag

כתבות (Article)

- כל כתבה כוללת כותרת, תוכן, תאריך פרסום, שם המחבר
- לכל כתבה יכולה להיות רשימת תגיות (לסינון כתבות לפי קטגוריה) הכתבות זמינות לכולם לצפייה.

Comment

תגובות (Comment)

במודלים יש קשרי גומלין לדוגמא: לכתבה יש תגיות

- לכל כתבה יכולים להיות תגובות שנכתבו על ידי משתמשים רשומים
- כל תגובה כוללת תוכן, שם המשתמש, תאריך תגובה המנהל יכול למחוק תגובות לא רצויות.

למנהל יש כתבות לכתבות יש תגובות ולתגובות יש תגובות

מודלים:

User

UserProfile

Tags

Post

Comment

PostUserLikes

כל משתמש יכול לעשות לייק ל"הרבה" כתבות
לכתבה יכולים לעשות "הרבה" לייקים

קשר של רבים-לרבים
בין משתמש לכתבה

מודלים:

Post

סטטוס:

draft

published

archived

Comment

תגובה לתגובה:

מודלים:

```
from django.core.validators import MinLengthValidator, RegexValidator

class Post(models.Model):
    author = models.ForeignKey(UserProfile, on_delete=models.CASCADE)
    title = models.CharField(max_length=100, unique=True, validators=[
        MinLengthValidator(5),
        RegexValidator(r'^[\w\s]+$', 'Title must be alphanumeric.')
    ])
    text = models.TextField(validators=[MinLengthValidator(5)])
    tags = models.ManyToManyField(Tag, blank=True)
    status = models.CharField(choices=status_choices, default='draft')
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f'Post: {self.title} by {self.author.user.username}'
```

מודלים:

```
class Comment(models.Model):
    post = models.ForeignKey(Post, on_delete=models.CASCADE)
    author = models.ForeignKey(UserProfile, on_delete=models.CASCADE)
    text = models.TextField(validators=[MinLengthValidator(5)], max_length=500)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    reply_to = models.ForeignKey('self', null=True, blank=True, on_delete=models.CASCADE)

    def __str__(self):
        return f'Comment by {self.author.user.username} on {self.post.title}'
```

מודלים:

```
LIKE_CHOICES = (  
    ('like', 'Like'),  
    ('dislike', 'Dislike'),  
)
```

```
class PostUserLikes(models.Model):  
    user = models.ForeignKey(UserProfile, on_delete=models.CASCADE)  
    post = models.ForeignKey(Post, on_delete=models.CASCADE)  
    like_type = models.CharField(choices=LIKE_CHOICES, default="like")  
    created_at = models.DateTimeField(auto_now_add=True)  
    updated_at = models.DateTimeField(auto_now=True)  
  
    # a user can't like a post twice  
    class Meta:  
        unique_together = ['user', 'post']  
  
    def __str__(self):  
        return f"{self.user.user.username} {self.like_type}d {self.post.title}"
```


הוספה לממשק ניהול ומיגרציות:

```
from django.contrib import admin

from api.models import Tag, UserProfile, Comment, Post, PostUserLikes

# each time we add a model:
admin.site.register([UserProfile, Tag, Comment, Post, PostUserLikes])

# python manage.py makemigrations api
# python manage.py migrate api
# python manage.py runserver

# visit http://localhost:8000/admin

# once per project:
# python manage.py createsuperuser
```


Serializers:

api/serializers.py

```
from api.models import Post, UserProfile, Tag, PostUserLikes, Comment
from django.contrib.auth.models import User
from rest_framework.serializers import ModelSerializer

# json + create/update + validations
class UserProfileSerializer(ModelSerializer):
    class Meta:
        model = UserProfile
        fields = '__all__'
        # fields = ['bio', 'id']

class TagSerializer(ModelSerializer):
    class Meta:
        model = Tag
        fields = '__all__'

class PostSerializer(ModelSerializer):
    class Meta:
        model = Post
        fields = '__all__'

class CommentSerializer(ModelSerializer):
    class Meta:
        model = Comment
        fields = '__all__'

class PostUserLikesSerializer(ModelSerializer):
    class Meta:
        model = PostUserLikes
        fields = '__all__'

class UserSerializer(ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email']
```

API Views:

api/views.py

```
from rest_framework.views import APIView

class Students(APIView):
    def get(self, request: Request) -> Response:
        return Response({"message": "GET"})

    def post(self, request: Request) -> Response:
        return Response({"message": "POST"})

class StudentsById(APIView):
    def get(self, request: Request, pk: int) -> Response:
        return Response({"message": "GET"})

    def post(self, request: Request, pk: int) -> Response:
        return Response({"message": "POST"})

    def delete(self, request: Request, pk: int) -> Response:
        return Response({"message": "DELETE"})

    def put(self, request: Request, pk: int) -> Response:
        return Response({"message": "PUT"})
```

ראינו בשיעור שעבר:

אפשר לכתוב קוד כדי להחזיר סטודנטים...

```
from api.views import say_hello, Students, StudentsById
from django.urls import path

urlpatterns = [
    path('hello/', say_hello),
    path('students/', Students.as_view(), name='students'),
    path('students/<int:pk>/', StudentsById.as_view(), name='student_detail'),
]
```

View Sets:

api/views.py

```
class StudentsViewSet(ViewSet):  
    def list(self, request: Request) -> Response:  
        return Response({"message": "GET"})  
  
    def create(self, request: Request) -> Response:  
        return Response({"message": "POST"})  
  
    def retrieve(self, request: Request, pk: int) -> Response:  
        return Response({"message": "GET"})  
  
    def update(self, request: Request, pk: int) -> Response:  
        return Response({"message": "PUT"})  
  
    def partial_update(self, request: Request, pk: int) -> Response:  
        return Response({"message": "PATCH"})  
  
    def destroy(self, request: Request, pk: int) -> Response:  
        return Response({"message": "DELETE"})
```

כל הפעולות במחלקה אחת:

```
from api.views import StudentsViewSet  
from rest_framework.routers import DefaultRouter  
  
router = DefaultRouter()  
router.register(r'students', StudentsViewSet, basename='students')  
urlpatterns = router.urls
```

Model View Sets:

api/views.py

```
from api.models import Tag
from api.serializers import TagSerializer

from rest_framework.viewsets import ModelViewSet

class TagViewSet(ModelViewSet):
    queryset = Tag.objects.all()
    serializer_class = TagSerializer
```

DRF תממש עבורכם את
כל הפעולות במחלקה אחת:

ModelViewSet שיורשת
מקבלת שאילתה
מקבלת סריליזר

ומשתמשת בהם עבורנו ליצור את כל הפעולות

api/urls.py

```
from api.views import TagViewSet
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register(r'tags', TagViewSet, basename='tags')
urlpatterns = router.urls
```

<http://localhost:8000/api/tags/>

Model View Sets:

api/urls.py

```
from api.views import TagViewSet, PostViewSet, CommentViewSet, PostUserLikesViewSet, UserProfileViewSet, UserViewSet
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register(r'tags', TagViewSet, basename='tags')
router.register(r'posts', PostViewSet, basename='posts')
router.register(r'comments', CommentViewSet, basename='comments')
router.register(r'post-user-likes', PostUserLikesViewSet, basename='post-user-likes')
router.register(r'user-profiles', UserProfileViewSet, basename='user-profiles')
router.register(r'users', UserViewSet, basename='users')
urlpatterns = router.urls
```

<http://localhost:8000/api/tags/>

יצירת דוקומנטציה OpenAPI 3

יאפשר לנו ליצור Postman Collection לכל הפרוייקט

<https://drf-spectacular.readthedocs.io/en/latest/readme.html#installation>

Installation

settings.py

Install using `pip`...

```
$ pip install drf-spectacular
```

then add drf-spectacular to installed apps in `settings.py`

```
INSTALLED_APPS = [  
    # ALL YOUR APPS  
    'drf_spectacular',  
]
```

and finally register our spectacular AutoSchema with DRF.

```
REST_FRAMEWORK = {  
    # YOUR SETTINGS  
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',  
}
```

drf-spectacular ships with sane [default settings](#) that should work reasonably well out of the box. It is not necessary to specify any settings, but we recommend to specify at least some metadata.

```
SPECTACULAR_SETTINGS = {  
    'TITLE': 'Your Project API',  
    'DESCRIPTION': 'Your project description',  
    'VERSION': '1.0.0',  
    'SERVE_INCLUDE_SCHEMA': False,  
    # OTHER SETTINGS  
}
```

Take it for a spin

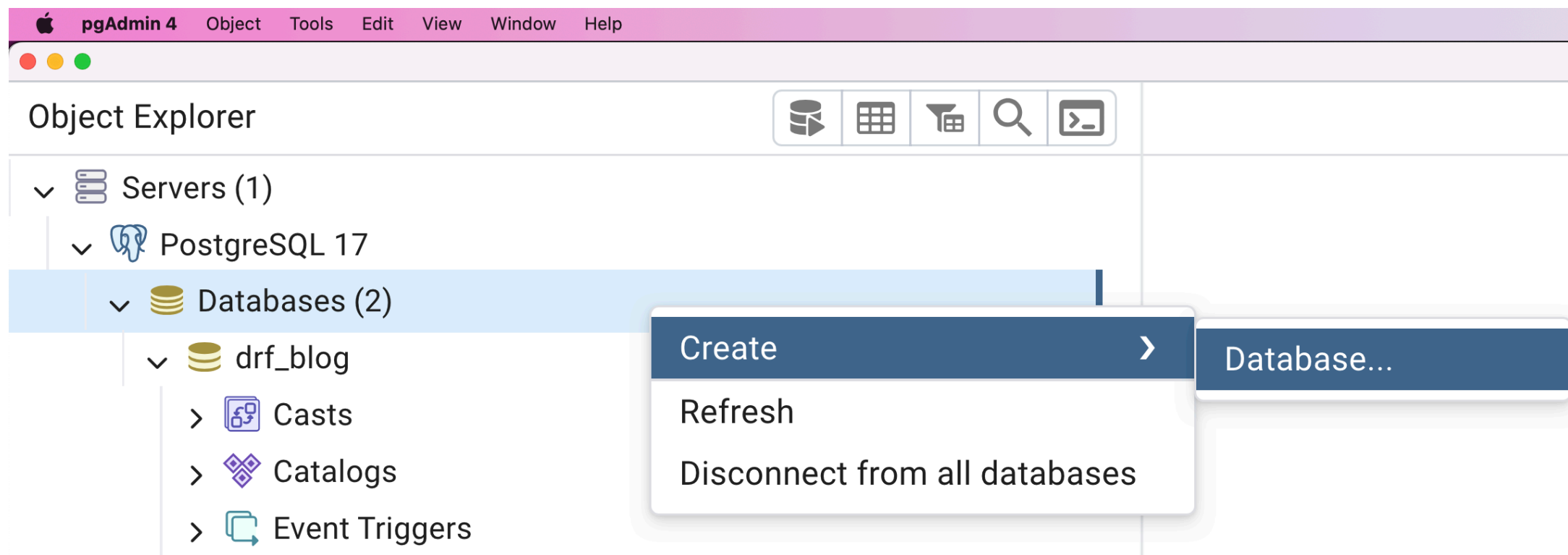
Generate your schema with the CLI:

```
$ ./manage.py spectacular --color --file schema.yml
```

יצירת דוקומנטציה OpenAPI 3

יאפשר לנו ליצור Postman Collection לכל הפרוייקט

Postgres SQL



דטה-בייס מובנה - טוב לבדיקות ולפיתוח...
בסופו של דבר נצטרך דטה-בייס אמיתי

user: postgres

password: 123456

Postgres SQL

```
pip install psycopg2
```

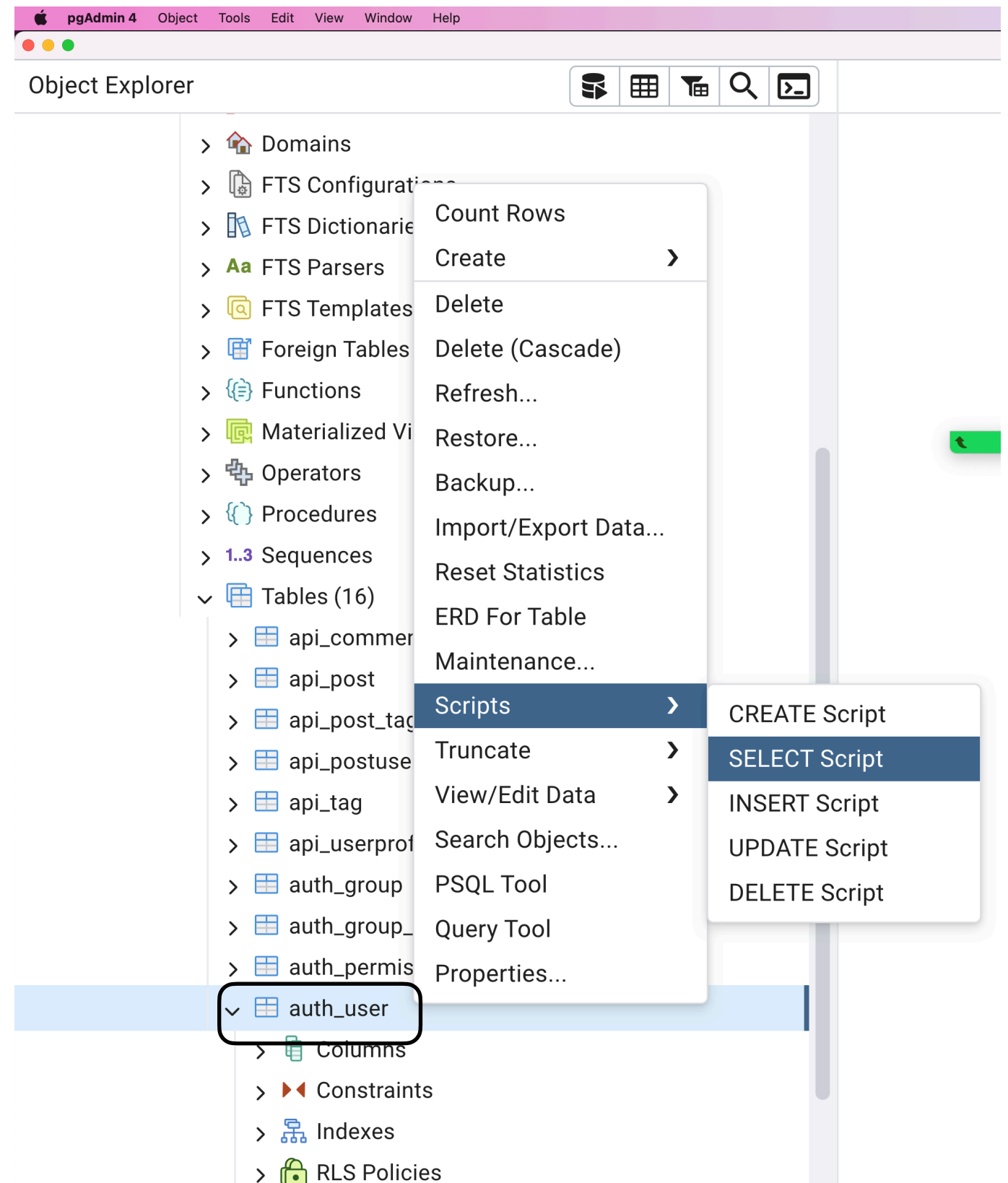
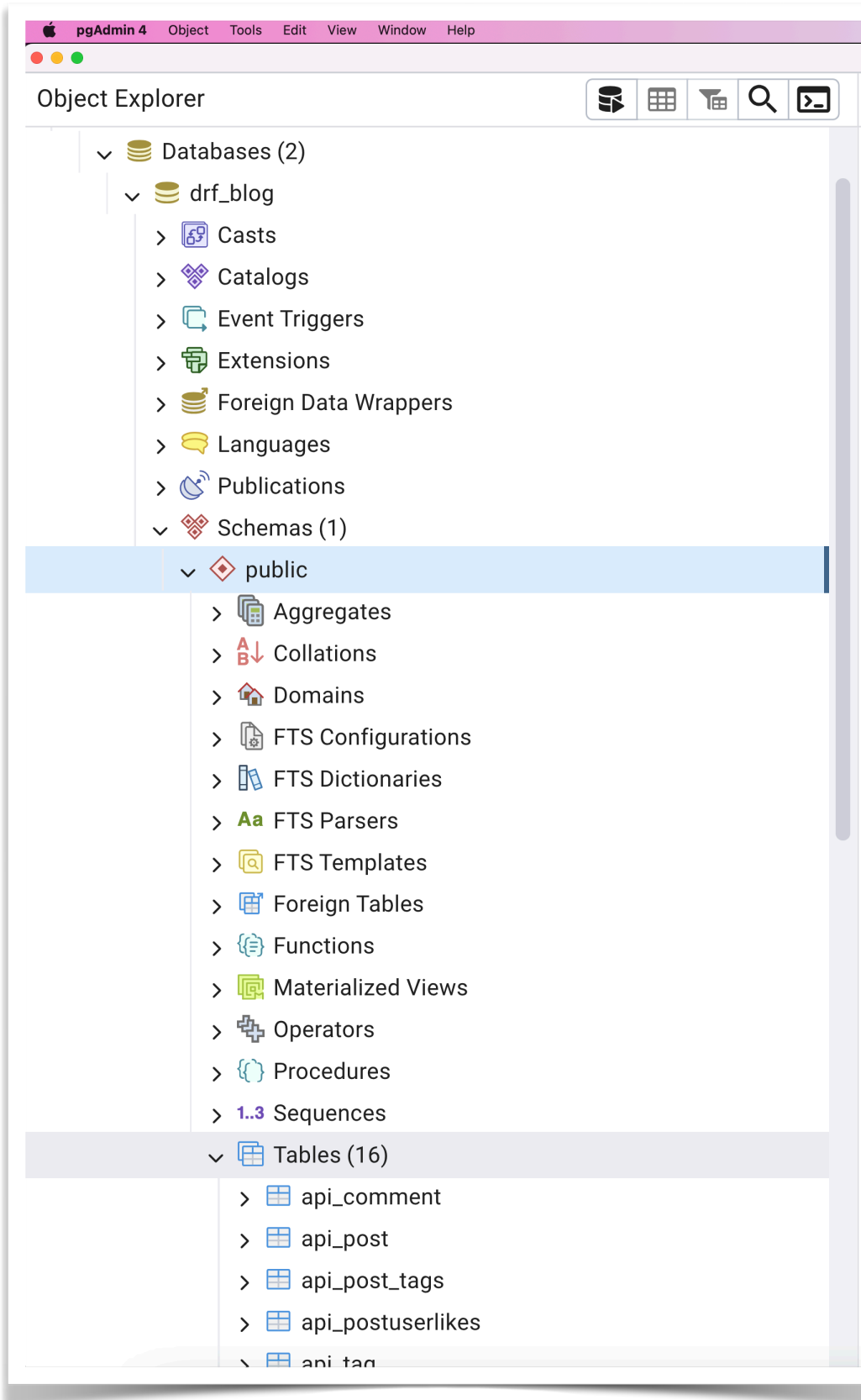
```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'drf_blog',  
        'USER': 'postgres',  
        'PASSWORD': '123456',  
        'HOST': 'localhost',  
        'PORT': '5432',  
    }  
}
```

```
python manage.py migrate
```

```
python manage.py createsuperuser
```

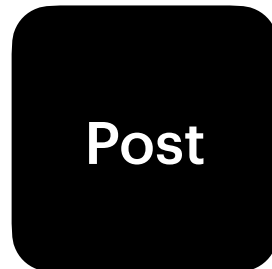
```
python manage.py createsuperuser --username admin --email tomer@gmail.com
```

Postgres SQL



זריעה של מסד הנתונים:

טבלה 1



טבלה 2



טבלה מקשרת - רבים לרבים



	id [PK] bigint	title character varying (100)	text text
1	1	ViewSets	After routing has

	id [PK] bigint	name character varying (32)
	1	Tech
	2	Python
	3	Typescript

post_id 1
tag_id 2

```
INSERT INTO api_tag(name)
VALUES('Cloud Deployment');
```

```
INSERT INTO api_post(title, text, status, created_at, updated_at, author_id)
VALUES('Happy Birthday to our Blog', 'yay!!hay', 'draft', '2025-09-04 15:33:29.523541+03',
'2025-09-04 15:33:29.523541+03', 1);
```

```
SELECT * FROM api_tag;
```

```
SELECT * FROM api_post; -- 10
```

```
SELECT * FROM api_tag; -- 8
```

```
INSERT INTO api_post_tags(post_id, tag_id)
VALUES(10, 8);
```

```
SELECT * FROM api_post_tags;
```

זריעה של מסד הנתונים:

```
INSERT INTO api_post(title, text, status, created_at, updated_at, author_id) VALUES
('REST API Design', 'Best practices for designing RESTful APIs including proper HTTP methods, status codes, and
resource naming conventions', 'draft', '2025-09-04 15:34:15.123456+03', '2025-09-04 15:34:15.123469+03', 1),

('Docker Containerization', 'Complete guide to containerizing applications with Docker, including Dockerfile
optimization and multi-stage builds', 'published', '2025-09-04 15:35:22.987654+03', '2025-09-04 15:35:22.987667+03',
1),

('Database Migrations', 'How to handle database schema changes safely in production environments using migration tools
and strategies', 'draft', '2025-09-04 15:36:08.555777+03', '2025-09-04 15:36:08.555790+03', 1),

('Authentication & JWT', 'Implementing secure user authentication using JSON Web Tokens, including token refresh
strategies and security considerations', 'published', '2025-09-04 15:37:41.333222+03', '2025-09-04
15:37:41.333235+03', 1),

('Testing Strategies', 'Comprehensive testing approach covering unit tests, integration tests, and end-to-end testing
for web applications', 'draft', '2025-09-04 15:38:55.888999+03', '2025-09-04 15:38:55.889012+03', 1),

('CI/CD Pipelines', 'Setting up continuous integration and deployment pipelines using GitHub Actions, including
automated testing and deployment', 'published', '2025-09-04 15:40:12.444555+03', '2025-09-04 15:40:12.444568+03', 1);
```

```
INSERT INTO api_tag(name)
VALUES('Cloud Deployment'); :זריעה של Many To Many
```

```
INSERT INTO api_post(title, text, status, created_at, updated_at, author_id)
VALUES('Happy Birthday to our Blog', 'yay!!hay', 'draft', '2025-09-04 15:33:29.523541+03', '2025-09-04
15:33:29.523541+03', 1);
```

```
INSERT INTO api_post_tags(post_id, tag_id)
VALUES(1, 1);
```